

ACI · AIMLCZG557

Local Search & Hill Climbing

Advanced Computing for AI — Lesson 3

Exam: 28 Jun 2026 · Closed book · Pseudocode expected

How to use this companion. Blue = the idea. Amber = everyday analogy. Green = fully-solved example. Red = common mistake. Purple = one line to remember.

1 What is Local Search?

Standard search (A^* , BFS) stores all paths it has explored. Local search is different: it only keeps **one current state** and moves to a neighbouring state. It never remembers where it has been. Because it stores only one state, memory cost is constant — it scales to problems with billions of states.

The intuition

Local search only cares about the final destination, not the journey. If you need to find the tallest mountain peak and you're already in the mountains, you don't care about the road you drove in on — you just want to climb higher and higher until you can't.

★ Everyday picture

Imagine adjusting the equaliser on a stereo. You try a bass setting, listen, then nudge one dial. If it sounds better, keep it. If not, try a different nudge. You never write down every combination you tried — you just keep improving from where you are.

When to use local search: when you only care about the final state (not how you got there) — N-Queens, VLSI layout, job scheduling, Travelling Salesman Problem.

Where this shows up in ML. Gradient descent is a continuous local search. Each parameter update is a move to a neighbouring point in weight space. Adam, SGD with momentum, and RMSProp are all variants of moving greedily in parameter space.

✓ Key takeaway

Local search = one current state, move to neighbour, repeat. $O(1)$ memory. Good when you only care about the final state.

2 Hill Climbing

Hill climbing is the simplest local search. At each step, it picks the neighbour with the highest value (steepest ascent). If no neighbour is better, it stops.

The intuition

You are blindfolded on a hillside. At each step, you feel around in all directions and step in whichever direction goes up the most. When every direction goes down, you stop. You are at a peak — but is it the highest peak in the whole range? You don't know.

Pseudocode (exam-ready):

```

function HILL-CLIMBING(problem):
    current = MAKE-NODE(INITIAL-STATE[problem])
    loop do
        neighbor = highest-valued successor of current
        if VALUE[neighbor] <= VALUE[current]:
            return STATE[current]
        current = neighbor

```

Steps: (1) start at initial state; (2) evaluate all neighbours; (3) move to best neighbour; (4) if no neighbour improves, return current; (5) else repeat.

Worked example: 4-Queens hill climbing trace

Board: 4 queens on a 4×4 board, one per column. State = {col1-row, col2-row, col3-row, col4-row}. Value = negative number of attacking pairs (higher value = fewer conflicts).

Step 1. Initial state: {2,4,1,3} (rows 2,4,1,3 for columns 1,2,3,4). Count attacks = 2. Value = -2.

Step 2. Evaluate all neighbours (move each queen to a different row in its column — $3 \times 4 = 12$ moves total). Best neighbour has value -1 (only 1 attack).

Step 3. Move to that neighbour. New state: {3,4,2,1}, value = -1.

Step 4. Evaluate neighbours again. Best neighbour has value 0 (no attacks).

Step 5. Move. New state: {2,4,1,3} ... wait, this is the start state! Stuck in a loop.

Result: Hill climbing got stuck in a cycle — a plateau. This illustrates why hill climbing is incomplete.

3 Problems with Hill Climbing

There are exactly three failure modes. Memorise all three.

- 1. Local maximum.** A state better than all neighbours, but not the global maximum. The algorithm stops here.
- 2. Plateau.** A flat area — all neighbours have the same value. The algorithm wanders randomly or stops.
- 3. Ridge.** A sequence of local maxima that are hard to navigate because the landscape is narrow. Stepwise moves go sideways, not up.

Watch out

Hill climbing is incomplete and not optimal. It will frequently get stuck at a local maximum. On the 8-Queens problem, steepest-ascent hill climbing gets stuck 86% of the time from a random start. The solution is to restart from a new random state (*random restart hill climbing*).

Key takeaway

Three failure modes: local maximum (stuck), plateau (wandering), ridge (narrow peak). Fix with random restarts.

4 Hill Climbing Variants

Stochastic hill climbing. Instead of always picking the best neighbour, pick a random uphill neighbour. Slower on average but escapes ridges better.

First-choice hill climbing. Generate neighbours randomly; take the first one that is better. Useful when there are many neighbours (large branching factor).

Random-restart hill climbing. Run hill climbing from multiple random starting states. Keep the best result. If each run has a probability p of finding the global optimum, then after k restarts the probability of success is $1 - (1 - p)^k$.

Where this shows up in ML. Random restarts are the inspiration behind random seeds, multiple training runs, and ensemble methods in ML. Running gradient descent from multiple random initialisations and taking the best result is exactly random-restart hill climbing.

✓ Key takeaway

Random restart = best practical fix for local maxima. Run k times; keep best. Success probability $\rightarrow 1$ as $k \rightarrow \infty$.

5 Simulated Annealing

Simulated annealing (SA) escapes local maxima by sometimes accepting a *worse* move. The probability of accepting a bad move decreases over time as the **temperature** T falls.

🔍 The intuition

When metal is annealed (heated then slowly cooled), the atoms settle into a low-energy crystalline structure. If cooled too fast, they freeze in a bad arrangement. SA mimics this: early on, the algorithm is “hot” and accepts many bad moves (lots of exploration). As temperature drops, it becomes increasingly picky (exploitation), settling into a good solution.

Acceptance probability. If the new state is better ($\Delta E > 0$), always accept. If worse ($\Delta E < 0$), accept with probability:

$$P(\text{accept}) = e^{\Delta E/T}$$

where $\Delta E = \text{value}(\text{new}) - \text{value}(\text{current})$ (negative for a bad move) and T is the current temperature.

📝 Worked example: SA acceptance calculation

Current value = 5. Neighbour value = 3. Temperature $T = 2$.

Step 1. $\Delta E = 3 - 5 = -2$ (bad move — value dropped).

Step 2. $P(\text{accept}) = e^{-2/2} = e^{-1} \approx 0.368$.

Step 3. Generate a uniform random number $r \in [0, 1]$. If $r < 0.368$, accept the bad move.

At $T = 0.5$: $P = e^{-2/0.5} = e^{-4} \approx 0.018$ — much less likely to accept. As $T \rightarrow 0$, SA behaves like ordinary hill climbing.

✗ Watch out

The cooling schedule matters enormously. Cool too fast: gets stuck in a local optimum. Cool too slow: wastes computation. A typical schedule is geometric: $T \leftarrow \alpha \cdot T$ per step, with $\alpha \approx 0.99$.

Where this shows up in ML. SA has been used to train neural networks (before gradient descent dominated). Its core idea — accept bad moves with a decaying probability — appears in Bayesian optimisation’s acquisition function design.

✓ Key takeaway

SA: always accept better; accept worse with $P = e^{\Delta E/T}$. As temperature $T \rightarrow 0$, SA $\rightarrow$ hill climbing. Optimal with slow enough cooling.

6 Local Beam Search

Local beam search runs k states in parallel instead of one. At each step, all k states generate their successors. Then the k best successors across *all* states are kept.

👉 The intuition

Instead of one hiker, send k hikers across the mountain range. At each step, each hiker explores their neighbourhood. Keep the k best positions found by anyone. Information is shared across hikers — a lucky hiker's discovery benefits everyone.

Unlike k independent random restarts, local beam search shares information. If one of the k states finds a promising region, successors in that region will dominate the next generation.

Stochastic beam search addresses the diversity problem: instead of keeping the top- k successors deterministically, they are chosen probabilistically (proportional to value), maintaining diversity.

Where this shows up in ML. Beam search in NLP decoding keeps the top- k partial sequences at each step. It is exactly local beam search over the token sequence space.

✓ Key takeaway

Local beam search = k parallel hill-climbers sharing information. Beam search in NLP is the same idea.